OFFICIAL USER MANUAL & REFERENCE

Centrix_LineFollower_Normal

Arduino C++ Library สำหรับหุ่นยนต์เดินตามเส้น RP2040

คู่มือฉบับสมบูรณ์: ขั้นตอนการติดตั้งเข้า Arduino IDE, แผนผังวงจรฮาร์ดแวร์, การปรับแต่งขาเซนเซอร์, สรุปฟังก์ชัน API ทั้งหมด, และเทมเพลตโค้ดสำหรับลงสนามแข่งขันจริง

บอร์ดไมโครคอนโทรลเลอร์ที่รองรับ: Raspberry Pi RP2040 / RP2040-Zero
เซนเซอร์: แผงเซนเซอร์อนาล็อก 16 ช่อง ผ่านไอซีมัลติเพล็กเซอร์ 74HC4067
ไดรเวอร์มอเตอร์: 2x DRV8871 H-Bridge Dual DC Motors
เวอร์ชันไลบรารี: v1.0.0 • ภาษาไทย • สไตล์ Direct API

1. บทนำและจุดเด่นของไลบรารี

ไลบรารี **Centrix_LineFollower_Normal** ถูกออกแบบมาโดยเฉพาะสำหรับนักเรียน นักศึกษา และนักประดิษฐ์หุ่นยนต์เดินตามเส้นความเร็วสูง โดยตัดความซับซ้อนของการเขียนแบบ Object-Oriented ออกไป แล้วใช้รูปแบบ **Direct API** (เหมือนไลบรารีชื่อดังอย่าง `pop32.h`) ทำให้คุณสามารถเรียกคำสั่งเช่น `Motor(50, 50);` หรือ `PD(60, 1.3, 16);` ได้ทันทีโดยไม่ต้องพิมพ์ `robot.` นำหน้า

จุดเด่นสำคัญ:

- **ไม่ต้องมี `robot.` นำหน้า:** สั้น กระชับ เขียนโค้ดสเต็ปในสนามแข่งได้รวดเร็ว
- **รองรับเซนเซอร์ 16 ช่อง:** คำนวณแบบ Centroid ละเอียดระดับทศนิยม ค่าตำแหน่ง `0 .. 15000`
- **ควบคุมความถี่คงที่ 1 kHz (1000 รอบ/วินาที):** ทำให้ตัวควบคุม PD ทำงานได้นิ่งและสม่ำเสมอที่สุด
- **ตรวจจับทางแยกอัจฉริยะ:** มีฟังก์ชันตรวจจับแยกเส้นตัด, แยกซ้าย, แยกขวา พร้อมระบบเลี้ยว 90 องศาในตัว
- **เปลี่ยนขา Pin ได้อิสระ:** มีคำสั่ง `setSensorPins()` และ `setMotorPins()` ในโค้ดหลัก

2. วิธีการติดตั้งเข้าสู่ Arduino IDE

1 ตรวจสอบโฟลเดอร์ไลบรารีของเครื่อง

เปิดโฟลเดอร์ Arduino Sketchbook ในเครื่องคอมพิวเตอร์ของคุณ:

- **macOS:** `/Users/<ชื่อผู้ใช้>/Documents/Arduino/libraries/`
- **Windows:** `C:\Users\<ชื่อผู้ใช้>\Documents\Arduino\libraries\`
- **Linux:** `~/Arduino/libraries/`

2 คัดลอกโฟลเดอร์ `Centrix_LineFollower_Normal`

นำโฟลเดอร์ `Centrix_LineFollower_Normal` ทั้งโฟลเดอร์ไปวางไว้ในโฟลเดอร์ `libraries` ดังกล่าว

โครงสร้างไฟล์ที่ถูกต้องจะเป็นดังนี้:

```
libraries/
└── Centrix_LineFollower_Normal/
    ├── Centrix_LineFollower_Normal.h
    ├── Centrix_LineFollower_Normal.cpp
    ├── keywords.txt
    ├── library.properties
    ├── README.md
    └── examples/ ...
```

3 รีสตาร์ท Arduino IDE และเปิดตัวอย่าง

ปิดโปรแกรม Arduino IDE แล้วเปิดขึ้นมาใหม่ จากนั้นไปที่เมนู:

```
File > Examples > Centrix_LineFollower_Normal > 07_Competition_Template
```

3. ผังการต่อวงจรฮาร์ดแวร์ (Hardware Pinout)

ไลบรารีได้กำหนด Pin ค่าเริ่มต้น (Default) ที่เหมาะสมกับบอร์ด **RP2040-Zero** ไว้ดังนี้:

อุปกรณ์ / หน้าที่	ขา RP2040-Zero	คำอธิบาย
Motor 1 IN1 / IN2	`GP0` / `GP1`	ขับมอเตอร์ซ้ายผ่าน DRV8871 (PWM 20 kHz Slow Decay)
Motor 2 IN1 / IN2	`GP2` / `GP3`	ขับมอเตอร์ขวาผ่าน DRV8871
Switch 1 (SW1)	`GP4`	สวิตช์ Calibrate แสง ขาว/ดำ (Active LOW, Internal Pullup)
Switch 2 (SW2)	`GP5`	สวิตช์ Start ออกตัว / รีสตาร์ทเมื่อกดฉุกเฉิน
RGB LED	`GP16`	ไฟ WS2812B NeoPixel บนบอร์ด RP2040-Zero
Mux SIG (Analog)	`GP26 (A0)`	ขาสัญญาณอนาล็อกรวมจากไอซี 74HC4067 (ADC 12-bit, 0..4095)
Mux S0, S1, S2, S3	`GP11, GP10, GP9, GP8`	ขาเลือกแชนแนลเซนเซอร์ CH0 ถึง CH15

4. การปรับแต่ง Pin และทิศทางในโค้ด (Custom Pins)

หากบอร์ดหุ่นยนต์ของคุณต่อขาไม่ตรงกับค่าเริ่มต้น หรือมอเตอร์หมุนกลับด้าน สามารถตั้งค่าใหม่ได้ใน `setup()` :

```
void setup() {
  // กำหนดขาเซนเซอร์ใหม่: setSensorPins(SIG, S0, S1, S2, S3);
  setSensorPins(26, 11, 10, 9, 8);

  // กำหนดขามอเตอร์ใหม่: setMotorPins(M1_IN1, M1_IN2, M2_IN1, M2_IN2);
  // setMotorPins(0, 1, 2, 3);

  // สลับทิศทางมอเตอร์กรณีล้อหมุนถอยหลัง
  LEFT_INVERT  = false;  // true = สลับมอเตอร์ซ้าย
  RIGHT_INVERT = false;  // true = สลับมอเตอร์ขวา

  // สลับด้านเซนเซอร์กรณีบอร์ดต่อกลับด้าน (CH0 อยู่ขวาสุด)
  SENSOR_REVERSE = false;

  // เลือกลักษณะสีเส้นและพื้น
  LINE_IS_BLACK  = true;  // true = เส้นดำพื้นขาว | false = เส้นขาวพื้นดำ

  initRobot(); // เริ่มต้นระบบหลังจากตั้งค่า
}
```

5. สรุปฟังก์ชัน API ทั้งหมดในไลบรารี

ฟังก์ชัน / ตัวแปร	พารามิเตอร์	รายละเอียดการทำงาน
`initRobot()`	-	ตั้งค่า Pin, PWM 20kHz, ADC 12-bit, LED, และโหลดค่าแสงจาก EEPROM
`waitForStart()`	-	แสดงไฟ Rainbow และรอให้กด SW2 เพื่อเริ่มวิ่ง (หรือกด SW1 Calibrate)
`Motor(left, right)`	`float, float` (-100..100)	สั่งขับมอเตอร์ซ้ายและขวาเป็นเปอร์เซ็นต์ (ติดลบคือถอยหลัง)
`Stop()`	-	เบรกหยุดมอเตอร์ทั้งสองข้างทันที
`position()`	-	อ่านตำแหน่งเส้น `0 .. 15000` (กึ่งกลาง = `7500`) หลุดเส้นจะจำทิศเดิม
`onLine`	ตัวแปร `bool`	สถานะว่าเซนเซอร์จับเส้นได้หรือไม่ (`true` / `false`)
`Calibrate()`	-	สั่งหุ่นยนต์หมุนตัวกวาดเส้นเพื่อเก็บค่าแสงขาว/ดำ และบันทึกลง EEPROM
`ShowSensor()`	-	พิมพ์ค่า ADC ดิบ 16 ช่อง (0..4095) ออก Serial Monitor สำหรับเทส
`ShowLine()`	-	พิมพ์ค่า Calibrated และตำแหน่งเส้นออก Serial Monitor
`PD(speed, Kp, Kd)`	`float, float, float`	เดินตามเส้น 1 สเต็ปที่ความถี่คงที่ 1 kHz (ใส่ในลูป)
`PD_Time(...)`	`speed, Kp, Kd, ms`	เดินตามเส้นตามเวลาที่กำหนดเป็นมิลลิวินาที (ms)
`PD_Cross(...)`	`speed, Kp, Kd, action`	วิ่งจนริมซ้ายและขวาเจอเส้นตัดพร้อมกัน แล้วทำ action
`PD_Left(...)`	`speed, Kp, Kd, action`	วิ่งจนริมซ้ายเจอเส้นตัวเดียว (แยกซ้าย) แล้วทำ action
`PD_Right(...)`	`speed, Kp, Kd, action`	วิ่งจนริมขวาเจอเส้นตัวเดียว (แยกขวา) แล้วทำ action
`turnLeft()`	-	หมุนตัวเลี้ยวซ้าย 90 องศาจนเซนเซอร์กลางจับเส้นใหม่ได้
`turnRight()`	-	หมุนตัวเลี้ยวขวา 90 องศาจนเซนเซอร์กลางจับเส้นใหม่ได้
`setLed(r, g, b)`	`uint8_t, uint8_t, uint8_t`	สั่งเปลี่ยนสีไฟ RGB NeoPixel (0..255)
`blinkLed(...)`	`r, g, b, count, ms`	สั่งกระพริบไฟ RGB ตามจำนวนครั้งและเวลา
`LedGreen() / LedRed()`	-	สั่งเปิดไฟสถานะสีเขียว / สีแดง
`pressed(pin)`	`uint8_t pin`	อ่านสถานะการกดปุ่ม (SW1, SW2) พร้อมระบบกันสัญญาณกระดอน (Debounce)

6. ฟังก์ชันจับแยกและพารามิเตอร์ Action

เมื่อหุ่นยนต์วิ่งด้วย PD จนกระทั่งเซนเซอร์ตัวริมตรวจพบเส้นตัดหรือทางแยก หุ่นยนต์จะหลุดออกจากลูป (Break) และทำคำสั่งตามตัวอักษร `action` ทันที:

Action	ชื่อเรียก	พฤติกรรมที่หุ่นยนต์ทำหลังจากเจอแยก
`'s' หรือ 'S'`	**Stop (หยุด)**	เบรกหยุดหุ่นยนต์ทันทีที่ตรวจพบเส้นตัด (ค่าเริ่มต้นหากไม่ระบุ)
`'l' หรือ 'L'`	**Turn Left (เลี้ยวซ้าย)**	เดินหน้าเล็กน้อยให้จุดหมุนตรงกับแยก แล้วหมุนซ้ายจนเซนเซอร์กลางจับเส้นใหม่ได้ แล้วหยุด
`'r' หรือ 'R'`	**Turn Right (เลี้ยวขวา)**	เดินหน้าเล็กน้อยให้จุดหมุนตรงกับแยก แล้วหมุนขวาจนเซนเซอร์กลางจับเส้นใหม่ได้ แล้วหยุด
`'p' หรือ 'P'`	**Pass (ข้ามแยก)**	วิ่งตรงข้ามเส้นตัดไปข้างหน้าจนพ้นแยก แล้วหยุดนิ่ง

7. ชุดตัวอย่างโค้ดใน Examples (7 หัวข้อ)

ในเมนู `File > Examples > Centrix_LineFollower_Normal` มีตัวอย่างพร้อมใช้งานดังนี้:

ชื่อตัวอย่าง	วัตถุประสงค์และการเรียนรู้
`01_TestMotor`	ทดสอบทิศทางมอเตอร์ เดินหน้า ถอยหลัง เลี้ยวซ้าย ขวา หมุนสปิน และเบรก
`02_TestLED`	ทดสอบไฟ RGB NeoPixel (สีพื้นฐาน, สีผสมอิสระ, กระพริบ, ไฟสีรุ้ง Rainbow)
`03_ReadSensor`	อ่านค่าดิบ ADC 16 ช่อง, อ่านตำแหน่งเส้น `position()` , และวิธีกำหนด Pin
`04_Calibration_EEPROM`	โหมด Calibrate แสงขาว/ดำ อัตโนมัติ บันทึก/โหลด EEPROM และพิมพ์ตาราง Range
`05_SimplePD`	ตัวอย่างการวิ่งตามเส้นแบบเรียบง่าย พร้อมคำแนะนำการปรับค่า Kp, Kd
`06_FullControl_Mission`	ตัวอย่างการจัดลำดับสเต็ปการวิ่งภารกิจครบวงจร (Soft Start → แยกซ้าย → แยกขวา → ข้ามแยก → เส้นชัย)
`07_Competition_Template`	**เทมเพลตมาตรฐานสำหรับลงแข่งจริง:** ตั้งขาเซนเซอร์ได้, มีโค้ด Debug เซนเซอร์, รวมสเต็ป PD ทุกรูปแบบ, และใช้ `Stop();` ใน loop

8. โค้ดเทมเพลตสำหรับลงสนามแข่งขันจริง

โค้ดด้านล่างนี้มาจากตัวอย่าง `07_Competition_Template.ino` ซึ่งออกแบบมาให้ปรับแต่งได้ง่ายที่สุดก่อนลงแข่ง:

```
#include <Centrix_LineFollower_Normal.h>

void setup() {
  // 1. กำหนดขาเซนเซอร์ (SIG, S0, S1, S2, S3)
  setSensorPins(26, 11, 10, 9, 8);

  // 2. ปรับแต่งค่าพารามิเตอร์หุ่นยนต์
  LINE_IS_BLACK    = true;   // true = เส้นดำพื้นขาว
  SENSOR_REVERSE   = false;  // สลับด้านเซนเซอร์
  LEFT_INVERT      = false;  // กลับทิศทางมอเตอร์ซ้าย
  RIGHT_INVERT     = false;  // กลับทิศทางมอเตอร์ขวา

  TURN_SPEED       = 50;     // ความเร็วเลี้ยว 90 องศา (%)
  CROSS_FORWARD_MS = 120;    // เดินหน้าก่อนเลี้ยว (ms)
  CROSS_PASS_MS    = 180;    // เดินหน้าข้ามแยก (ms)

  // 3. เริ่มต้นระบบและโหลดค่า Calibrate จาก EEPROM
  initRobot();

  // 4. ตรวจเช็คเซนเซอร์ก่อนแข่ง (แข่งจริงให้ใส่ // ปิดไว้)
  // ShowSensor();
  // ShowLine();

  // 5. รอสวิตช์ SW2 ออกตัว (หรือกด SW1 เพื่อ Calibrate แสง)
  waitForStart();
  LedGreen();

  // 6. ลำดับภารกิจในสนามแข่ง (เลือกเปิดคำสั่งที่ต้องใช้)
  // [Step 1] ออกตัวนุ่มนวล
  PD_Time(55, 1.2, 14, 1200);

  // [Step 2] วิ่งจนเจอแยกซ้าย แล้วเลี้ยวซ้าย 90 องศา
  PD_Left(60, 1.3, 16, 'l');

  // [Step 3] วิ่งจนเจอแยกขวา แล้วเลี้ยวขวา 90 องศา
  PD_Right(60, 1.3, 16, 'r');

  // [Step 4] วิ่งข้ามทางแยกตัดตรงไปข้างหน้า (ไม่เลี้ยว)
  PD_Cross(60, 1.3, 16, 'p');

  // [Finish] เข้าเส้นชัยสุดท้าย (เจอเส้นตัดแล้วเบรกหยุดสนิท)
  PD_Cross(60, 1.3, 16, 's');

  Stop();
  blinkLed(0, 255, 0, 5, 120); // ฉลองจบภารกิจ
}

void loop() {
  // เมื่อจบภารกิจใน setup() แล้ว ให้เบรกหยุดมอเตอร์ตลอดเวลา
  Stop();
}
```

9. ข้อแนะนำและการแก้ปัญหา (Troubleshooting)

1. มอเตอร์หมุนถอยหลัง หรือเลี้ยวผิดทิศ:
ให้เปลี่ยนค่า `LEFT_INVERT = true;` หรือ `RIGHT_INVERT = true;` ใน `setup()`

2. หุ่นยนต์ส่ายไปมาขณะวิ่งตามเส้น:
แสดงว่าค่า `Kp` สูงเกินไป ให้ลดค่า Kp ลงเล็กน้อย (เช่น จาก 1.5 เหลือ 1.2) หรือเพิ่มค่า `Kd` (เช่น จาก 14 เป็น 18) เพื่อเพิ่มแรงหน่วง

3. หุ่นยนต์หลุดโค้ง เลี้ยวไม่ทัน:
แสดงว่าค่า `Kp` ต่ำเกินไป หรือความเร็ว `speed` สูงเกินไปสำหรับรัศมีโค้ง ให้เพิ่ม Kp หรือลดความเร็วลง

4. ปุ่มหยุดฉุกเฉินขณะวิ่ง (Emergency Stop):
ระหว่างที่หุ่นยนต์กำลังวิ่ง หากกดปุ่ม `SW1` หรือ `SW2` ตัวหุ่นยนต์จะเบรกหยุดทันทีและกระพริบไฟสีแดงเตือน สามารถกด `SW2` อีกครั้งเพื่อรีสตาร์ทบอร์ด